Thinking 2.1 请根据上述说明，回答问题：在编写的 C 程序中，指针变量中存储的地址被视为虚拟地址，还是物理地址？MIPS 汇编程序中 `lw` 和 `sw` 指令使用的地址被视为虚拟地址，还是物理地址？

1.虚拟地址

2.虚拟地址

Thinking 2.2 请思考下述两个问题：

- 从可重用性的角度，阐述用宏来实现链表的好处。
- 查看实验环境中的 `/usr/include/sys/queue.h`，了解其中单向链表与循环链表的实现，比较它们与本实验中使用的双向链表，分析三者在插入与删除操作上的性能差异。

使用宏实现的链表（如SLIST、STAILQ、CIRCLEQ）可以减少代码重复，提高效率，并降低错误率。

性能差异：

1. 单向链表的插入操作的时间复杂度为O(1)，删除操作的时间复杂度为O(n)；
2. 循环链表的插入操作的时间复杂度为O(1)，删除操作的时间复杂度为O(n)；
3. 双向链表的插入操作的时间复杂度为O(1)，删除操作的时间复杂度为O(1)；

Thinking 2.3 请阅读 `include/queue.h` 以及 `include/pmap.h`，将 `Page_list` 的结构梳理清楚，选择正确的展开结构。

```
A:
struct Page_list{
    struct {
        struct {
            struct Page *le_next;
            struct Page *le_prev;
        } pp_link;
        u_short pp_ref;
    }* lh_first;
}
```

```
B:
struct Page_list{
    struct {
        struct {
            struct Page *le_next;
            struct Page **le_prev;
        } pp_link;
        u_short pp_ref;
    } lh_first;
}
```

```
C:
struct Page_list{
    struct {
        struct {
            struct Page *le_next;
            struct Page **le_prev;
        } pp_link;
        u_short pp_ref;
    }* lh_first;
}
```

正确的为C

> **Thinking 2.4** 请思考下面两个问题：
>
> - 请阅读上面有关 TLB 的描述，从虚拟内存和多进程操作系统的实现角度，阐述 ASID 的必要性。
> - 请阅读 MIPS 4Kc 文档《MIPS32® 4K™ Processor Core Family Software User's Manual》的 Section 3.3.1 与 Section 3.4，结合 ASID 段的位数，说明 4Kc 中可容纳不同的地址空间的最大数量。

ASID是TLB（转换后备缓冲器）中用于标识进程的额外位，允许TLB条目区分不同进程，从而容纳多个进程的地址映射。这意味着即使多个进程有相同的虚拟地址映射到不同的物理地址，只要它们的ASID不同，这些映射就可以共存。ASID通常是6位，因此最多可以区分64个不同的进程空间

> **Thinking 2.5** 请回答下述三个问题：
>
> - tlb_invalidate 和 tlb_out 的调用关系？
> - 请用一句话概括 tlb_invalidate 的作用。
> - 逐行解释 tlb_out 中的汇编代码。

tlb_invalidate调用了tlb_out。

在页表更新时更新TLB。

首先将CP0_ENTRYHI寄存器的值保存到k1寄存器，接着把a0寄存器的值复制到k1寄存器中。

执行mtc0指令以关闭流水线，然后运行tlbp指令进行TLB搜索，以确保所需的参数地址已在TLB中注册。如果地址匹配， tlbp会在Index寄存器中返回相应的索引值； 如果未找到匹配项， 则Index寄存器将返回最大索引值加一。

由于流水线的存在，写入Index寄存器后需要插入四条nop指令以确保数据稳定，这是因为五级流水线的特性要求暂停以待指令执行。

将Index寄存器的值读取到k0寄存器，检查此值是否为负。如果k0显示负值，说明未在TLB找到匹配项，程序将跳转到NOFOUND标签处理。否则，表示找到有效的TLB条目，随后会清空HI和LO寄存器，使用找到的索引加载相应的ENTRYHI和ENTRYLO值。

k1 的值再次存储在 HI 寄存器中，然后返回。

> **Thinking 2.6** 从下述三个问题中任选其一回答：
>
> - 简单了解并叙述 X86 体系结构中的内存管理机制，比较 X86 和 MIPS 在内存管理上的区别。
> - 简单了解并叙述 RISC-V 中的内存管理机制，比较 RISC-V 与 MIPS 在内存管理上的区别。
> - 简单了解并叙述 LoongArch 中的内存管理机制，比较 LoongArch 与 MIPS 在内存管理上的区别。

差别主要在对TLB不命中的处理上。

MIPS与RISC-V的较大区别之一是除相同不同外，是否具有不同条件的条件分支。RISC-V中有单纯比较两个寄存器的分支指令，但MIPS中有根据比较结果是否为真，将寄存器设定为0或1的比较指令。